Flaky Tests in the AI Domain

Péter Attila Soha*, Béla Vancsics*, Tamás Gergely*, Árpád Beszédes*
*Department of Software Engineering, University of Szeged
Szeged, Hungary
{psoha,vancsics,gertom,beszedes}@inf.u-szeged.hu

ABSTRACT

In this position paper, we investigate how frequently is randomness the cause of flakiness in the traditional and in the AI-enabled software domains. Based on previous works, it seems that while in the general domain flakiness rarely stems from randomness, in the AI domain it is a frequent phenomenon. Thus, we urge a discussion about a classification scheme of flaky tests based on whether they are caused by the inherent randomness of the AI-enabled SUT or some other reason. This way, better identification, classification and proper handling of flakiness in such systems will be possible.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; • **Theory of computation** → *Pseudorandomness and derandomization*; • **Computing methodologies** → Artificial intelligence; Machine learning.

KEYWORDS

Flaky test, artificial intelligence, machine learning, randomness

ACM Reference Format:
Péter Attila Soha[1], Béla Vancsics[1], Tamás Gergely[1], Árpád Beszédes[1]. 2024. Flaky Tests in the AI Domain. In *2024 International Flaky Tests Workshop 2024 (FTW '24), April 14, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3643656.3643897

1 INTRODUCTION

Test flakiness is a well-studied problem in software testing [1], and occurs when a test case executed multiple times in (seemingly) the same way can either pass or fail (seemingly) randomly. In the literature, there are several reasons identified for a test to be flaky, and several actions proposed to handle flakiness. One of the reasons is the random nature of the system under test (SUT). In this work, we concentrate on a particular class of software in which randomness is inherently present: AI/ML-enabled software.

Most AI/ML algorithms include various sources of randomness, posing a high risk for their tests to demonstrate flakiness, probably to a higher degree than traditional software [2, 3]. Yet, only a few studies have addressed the test flakiness problem in the context of AI/ML. Also, empirical comparison of flakiness between AI/ML and traditional software is mostly missing from the literature.

The root causes of flakiness in AI/ML systems can be principally divided as (a) reasons that are the consequence of the inherent randomness of the AI/ML software, and (b) other, more technical causes which are independent of the AI/ML aspect and should be dealt with using known techniques. The flakiness of tests in the first group cannot be completely eliminated, but by using better suited tests (such as less strict oracles or approximate assertions) their prevalence can be mitigated [3–5]. The main goal of our ongoing research is to find ways to automatically detect the causes of test flakiness in AI/ML systems, and to devise approaches for designing tests for such systems, which are more resilient to flakiness.

2 RELATED WORK

Randomness as the cause for flakiness has been studied numerous times for traditional software. In their study on flakiness root causes [6] Eck et al. confirmed about randomness that it shows a "very low diffusion" as a cause. A similar conclusion can be drawn from the work of Parry et al. [7], who conducted a survey among developers not restricted to the AI domain. They found that randomness is one of the least probable reasons for flakiness. Some of the respondees indicated that "the term flaky is inappropriately used to blame test cases when their flakiness is inevitable if they test non-deterministic code".

However, the situation is different when we consider AI/ML applications. Dutta et al. performed a study on flaky tests in probabilistic and ML applications [2]. They identified several causes of flakiness, the main one being the random nature of probabilistic and ML algorithms. In 60% of the 75 cases they examined, randomness caused flakiness, and in 40 cases (53%) the root cause was the inherent randomness of the ML algorithm.

One way to overcome the flakiness of tests caused by the inherent randomness of the SUT is to set seeds for random generation. Dutta et al. [3] carried out an empirical analysis of 114 ML projects with 2226 flaky tests, 426 of which they were able to fix by seeding, and another 42 by using other means (adjusting hyper-parameters, changing assertion bounds, refactoring the test, and fixing the code).

Arcuri et al. [5] proposed several statistical methods to properly assess the results of randomized algorithms. These methods could be used to reduce the flakiness of test cases.

Dutta et al. [4] presented a tool for automatically fixing flaky tests due to algorithmic randomness in ML algorithms. This tool can fix the tests that use approximate assertions to compare the test results and expected values. The basis of the technique is to estimate the tail distribution using the results of several runs and to modify the expected value ranges. However, in some cases, the expected value suggested by the tool may differ greatly from the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FTW '24, April 14, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0558-8/24/04…$15.00
https://doi.org/10.1145/3643656.3643897

previously specified value, which may reduce the fault detection ability of the test.

Xia et al. [8] investigated the prevalence and challenges of oracle approximation (OA) in testing deep learning (DL) libraries. They revealed that a significant portion of assertions utilizes OA, often requiring developers to modify the oracle or the output, or to use different approximation APIs and thresholds. The authors identified common sources of oracles and explored the maintenance and evolution challenges in OA practices.

Nejadgholi et al. [9] introduced FASER, a systematic approach for optimizing the balance between fault-detection effectiveness and test flakiness in ML projects. Addressing the inherent non-determinism in ML algorithms, FASER frames the optimization of assertion bounds as a trade-off problem, utilizing statistical methods for estimating the flakiness rate and mutation testing for fault detection effectiveness. Evaluated on 87 flaky tests from 22 prominent ML projects, FASER successfully identified 23 tests with overly conservative bounds. The proposed tighter bounds by FASER enhance fault-detection while controlling flakiness, with 14 out of 19 suggested modifications accepted by developers, showcasing FASER's practical impact in improving ML testing practices.

3 DISCUSSION

Several studies with traditional software have shown that randomness of the SUT is one of the least probable causes of a test being flaky [7, 10]. This is in high contrast with the results of other experiments made with AI systems, for example, Duta et al. [2], where they found that 60% of the flaky tests were caused by randomness or non-determinism.

Furthermore, some respondents of the survey by Perry et al. [7] indicated that flakiness can indicate buggy test code as well as non-deterministic production code. Luo et al. [10] also found that 24% of the analyzed commits fix the flakiness by changing both the tests and the code under test, which indicates that, in addition to fixing the test, it was also necessary to change the production code. However, in some cases, flakiness cannot be avoided this way due to the non-deterministic nature of the SUT.

Based on this, we believe that an important research direction should be to investigate whether the flakiness of tests in AI-enabled systems is caused by the inherent non-deterministic (random) nature of the production code (which is a fundamental nature of AI/ML algorithms), or by something else. In the former case, flakiness cannot be eliminated but tests can be reworked to mitigate the frequency of its occurrences. As we have seen from the example citations in the previous section, there are methods proposed to mitigate the consequences of such randomness. Specific testing methods such as statistical testing [5] could offer better solutions than others (e.g. seeding [3]), because the latter eliminates inconvenient randomness but the expected results determined by some oracle will only be valid for that single seed, so the test case cannot really fulfill its purpose.

Derived from these observations, we identified several open questions regarding flakiness in the AI domain, which we also offer as topics for the workshop:

(1) How realistic is it to divide causes of flakiness into two categories and try to detect them automatically: (a) inherent randomness of the SUT caused by the underlying AI/ML algorithms, in which case the mitigation of flakiness can be performed by specific testing techniques, (b) other causes of randomness of the SUT and/or tests suite, which should be eliminated using other, known techniques (or ignored)?
(2) What are the patterns and/or criteria that can most likely indicate test cases of AI/ML-enabled systems that are not properly implemented (from the point of view of flakiness)? Could these cases be identified *without* a thorough manual examination of the flaky test and the production code?
(3) What methods of mitigating inherent flakiness are acceptable without drastically reducing the effectiveness and/or usefulness of the test case? When random is an inherent phenomenon of the SUT, can it be safely eliminated without corrupting the tests or the code?
(4) Finally, in which cases should we look to fixing the production code of the AI/ML-based system (including the traditional code, the learning data, the learning model, its metaparameters, etc) instead of trying to fix flaky tests?

REFERENCES

[1] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 1, oct 2021. [Online]. Available: https://doi.org/10.1145/3476105%7D

[2] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic, "Detecting flaky tests in probabilistic and machine learning applications," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2020. New York, NY, USA: Association for Computing Machinery, 2020, pp. 211–224. [Online]. Available: https://doi.org/10.1145/3395363.3397366

[3] S. Dutta, A. Arunachalam, and S. Misailovic, "To seed or not to seed? an empirical analysis of usage of seeds for testing in machine learning projects," in *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 151–161. [Online]. Available: https://doi.org/10.1109/ICST53961.2022.00026

[4] S. Dutta, A. Shi, and S. Misailovic, "Flex: Fixing flaky tests in machine learning projects by updating assertion bounds," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, pp. 603–614. [Online]. Available: https://doi.org/10.1145/3468264.3468615

[5] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd International Conference on Software Engineering*, ser. ICSE '11. New York, NY, USA: Association for Computing Machinery, 2011, pp. 1–10. [Online]. Available: https://doi.org/10.1145/1985793.1985795

[6] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019. New York, NY, USA: Association for Computing Machinery, 2019, pp. 830–840. [Online]. Available: https://doi.org/10.1145/3338906.3338945

[7] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "Surveying the developer experience of flaky tests," in *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 253–262. [Online]. Available: https://doi.org/10.1145/3510457.3513037

[8] C. S. Xia, S. Dutta, S. Misailovic, D. Marinov, and L. Zhang, "Balancing effectiveness and flakiness of non-deterministic machine learning tests," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1801–1813. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00154

[9] M. Nejadgholi and J. Yang, "A study of oracle approximations in testing deep learning libraries," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 785–796. [Online]. Available: https://doi.org/10.1109/ASE.2019.00078

[10] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, pp. 643–653. [Online]. Available: https://doi.org/10.1145/2635868.2635920

Received 7 December 2023